

COMPUTING SCIENCE I

2026-III Semester

Checkpoint Week 8 — Snake Battle

Group 6

Miguel Angel Quintero Puentes

Brayan Esteban Bonilla Piñeros

Luis Felipe Zubieta Rincón

Suhayl Mariana Barrantes Alvarado

Systems Engineering Program

School of Engineering

Universidad Distrital Francisco José de Caldas

1. GAME RULES AND MECHANICS

1.1. General description of the game

Snake Battle is a two-player video game developed in Python. The game involves controlling two snakes that move simultaneously within a grid board. One snake is controlled by the player and the other by an algorithm. The main goal is to survive on the board while collecting food that allows the snake to grow in size. The game ends when one or both snakes are eliminated due to a collision. If only one remains active, it will be declared the winner; if both are eliminated during the same cycle, the game will end in a tie.

1.2. Rules of the game

1.2.1. Representation of a snake's body

For the internal representation of each snake, a data structure based on an ordered list of positions on the board will be used. Each body segment will be stored as a pair of coordinates (x, y), where x represents the horizontal position and y represents the vertical position within the game space.

The list will maintain the order of the segments from the head to the tail of the snake. The first stored position will always correspond to the head, since it is the element that determines movement, while the last position will correspond to the tail.

An example of a snake's body representation is:

[(5,5),(5,6),(5,7)]

In this structure, position (5,5) represents the snake's head, the intermediate positions represent the body, and (5,7) represents the tail.

1.2.2. Movement of the snakes:

Each snake moves in a specific direction on the board (x,y)

The possible directions are:

- Up: the snake decreases its vertical position by one unit. Its movement corresponds to (0, -1).

- Down: the snake increases its vertical position by one unit. Its movement corresponds to (0, 1).
- Left: the snake decreases its horizontal position by one unit. Its movement corresponds to (-1, 0).
- Right: the snake increases its horizontal position by one unit. Its movement corresponds to (1, 0).

A snake cannot directly turn in the opposite direction of its current movement if doing so would cause its head to immediately enter the next segment of its own body.

Therefore:

- Up won't be able to switch directly to Down.
- Down won't be able to switch directly to Up.
- Left won't be able to switch directly to Right.
- Right won't be able to switch directly to Left.

1.2.3. Update the movement

The movement of the snakes takes place within an update cycle where both snakes calculate their next position before changing the state of the board.

The general update process is:

1. The direction selected by the player is received.
2. The AI calculates the next move for its snake.
3. The new position of both snakes' heads is determined.
4. The positions of both snakes are updated within the same cycle.
5. It is determined if any snake has reached a position occupied by food.
6. Collisions that occurred during movement are processed.
7. If a snake remains active and has eaten food, its growth and score are updated.
8. The eaten food is removed from foods.
9. New food is generated in a valid position to keep three foods active.
10. The overall game state is updated.

The update of each snake's body is done by modifying the list that represents its segments.

The procedure consists of:

1. Calculate the new head position by applying the displacement corresponding to the current direction.
2. Add the new position at the beginning of the list.
3. Check if the new position matches any food.
4. If the snake didn't eat food, remove the last element of the list.
5. If the snake ate food and remains active after processing collisions, keep the last segment, allowing its length to increase by one.

Example:

If a snake initially is at:

[(5,5),(5,6),(5,7)]

and makes a move to the right, the new position of the head will be:

(6,5)

After adding the new head, the list becomes:

[(6,5),(5,5),(5,6),(5,7)]

If the snake did not eat food, the last position corresponding to the tail is removed:

[(6,5),(5,5),(5,6)]

If food is at (6,5) and the snake survives the collision cycle, the tail is kept:

[(6,5),(5,5),(5,6),(5,7)]

This way, both normal movement and growth are simulated.

1.2.4. Growth of the snake

Growth happens when the head of a snake matches the position of any of the available food items, and the snake stays active after checking the cycle collisions.

When this condition is met, the last body segment is kept during the update.

The procedure is:

1. Add the new position of the head.
2. Check if the head reached any food.
3. Handle the collisions for this cycle.
4. If the snake is still active, keep the last segment of the body.
5. Increase the snake's score by one.
6. Remove the eaten food from foods.
7. Generate new food in a valid position.
8. Add the new food to foods.

In this way, after each consumption, a constant amount of three active foods is maintained at the same time, as long as there are enough free spots on the board.

A longer snake takes up more positions and, therefore, increases the movement restrictions available.

1.2.5. Food generation

The food represents a set of dynamic elements distributed across the board that allow the snakes to grow in size.

During the game, there will be three foods available at the same time, each placed in a different position. The positions will be generated randomly within the established boundaries of the board.

Each new position must be free. It cannot coincide with:

- No segment of the player's snake.

- No segment of the AI-controlled snake.
- Other existing food.

The generation process consists of:

1. Select a random position on the board.
2. Check that the position isn't occupied by any snake.
3. Make sure there isn't another food in the same spot.
4. If the position is taken, generate a new one.
5. If the position is free, add the new food to foods.

When a snake eats food and stays active, the food is removed and a new valid position is generated to replace it.

This way, three pieces of food remain active during the game.

1.2.6. Collision system

Collisions represent the main way of elimination in the game.

Since both snakes move at the same time, the new positions of their heads will be calculated before determining the outcome of the round.

The following types of collision are considered:

- **Collision with the edges of the board**

A snake loses when the position of its head goes beyond the established boundaries of the board. Conceptually:

$$x < 0$$

$$x \geq \text{board.width}$$

$$y < 0$$

$$y \geq \text{board.height}$$

- **Snake colliding with its own body**

A snake is eliminated when its head occupies a position corresponding to any of the remaining segments of its own body.

The current head should not be included in this check.

Conceptually:

$$\text{head in body}[1:]$$

- **Collision with the opposite snake**

If there is no direct collision between both heads, a snake will be eliminated when its head matches any of the segments of the rival snake's body.

For this check, the rival's head is not considered, since head-on collisions are handled as a separate case.

Conceptually:

```
player.head in AI.body[1:]
```

- **Head collision**

Since both snakes update simultaneously, it can happen that both heads try to occupy the same position on the board.

If: `player.head == AI.head`

Both snakes will be eliminated, and the game result will be a tie.

In this case, the length of the snakes will not be taken into account.

Head-to-head collision will take priority over checking for head-to-opponent-body collisions.

This prevents the same situation from being interpreted simultaneously as two different types of collision.

- **Simultaneous collisions**

Due to the simultaneous movement, it can also happen that each snake suffers a different collision during the same cycle.

For example:

- One snake can collide with a wall while the other collides with its own body.
- Both snakes can collide with different segments.
- Both heads can occupy the same position.

If both snakes are eliminated during the same update cycle, the game will end in a tie.

1.2.7. Victory and ending conditions

The main goal of Snake Battle is to stay active on the board while the rival snake gets eliminated.

The game can end under the following conditions:

- If the snake controlled by the player is eliminated and the AI remains active, the artificial intelligence wins.
- If the snake controlled by the AI is eliminated and the player remains active, the player wins.
- If both snakes are eliminated during the same update cycle, the game ends in a tie.
- If both snakes remain active, a new cycle begins.

The game's internal state will allow the result to be communicated to the graphical interface.

The states will be:

- **READY:** Initial state where the game is ready to start.
- **RUNNING:** State where the game is running and both snakes can move.
- **PAUSE:** State triggered by a player action. The positions of elements don't update while the game is paused.
- **PLAYER_WIN:** Terminal state where the player's snake is still active and the AI snake has been eliminated.
- **AI_WIN:** Terminal state where the AI-controlled snake is still active and the player's snake has been eliminated.
- **DRAW:** Terminal state that occurs when both snakes are eliminated during the same update cycle.

An additional GAME_OVER state won't be used, since PLAYER_WIN, AI_WIN, and DRAW already directly represent the possible final states of the game.

1.2.8. Overall flow of a match

The flow of a game is described as the following way:

1. Game initialization

During this stage:

- The board size is set.
- The starting positions of both snakes are assigned.
- The initial movement direction of each snake is set.
- The three initial foods are generated.
- The scores are initialized.
- The state READY is set.

The three meals must appear in valid positions, without overlapping with any snake or each other

2. Movements capture

When the game is in the RUNNING state, the system receives the player's input.

At the same time, the AI analyzes the current state.

The AI will use BFS to calculate a path to each available food and then Greedy to select the reachable path with the fewest moves.

Both decisions are made before changing the snakes' positions.

3. Positions update

For each snake:

- The new head position is calculated.
- It is added to the beginning of the body.
- It is determined whether the new position corresponds to food.

- If there is no consumption, the tail is removed.
- If there is a possible consumption, the final growth will be processed after determining that the snake survived the cycle.

4. Collision detection

Is checked:

- Collision between both heads.
- Collision against the boundaries.
- Collision against your own body.
- Collision against the rival snake's body.

If both snakes are eliminated during the same round, the result will be a DRAW.

5. Food processing

For each snake that stays alive:

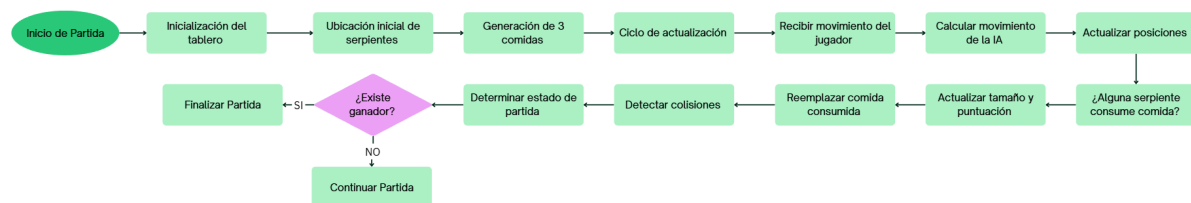
- It checks if it reached any food.
- Its length increases.
- Its score goes up.
- The corresponding food is removed from foods.
- A new food is generated in a valid position.

6. Game status update

After processing the cycle:

- Both active: RUNNING.
- Only player active: PLAYER_WIN.
- Only AI active: AI_WIN.
- None active: DRAW.

General update flow of a game entry



2. PSEUDOCODE OF DATA STRUCTURES AND ALGORITHMS

This section explains how the game's AI works and how the relevant algorithms are used and integrated within its operation. Each part of the algorithm and its specific functionality is explained in detail, in order to understand the development of the game's internal structure and the application of important topics within it.

2.1. General functioning of AI

The snake controlled by the computer that faces the player has to make a decision each time a movement cycle in the game runs. Based on the current state of the board and the snake's current position, the AI checks the available food on the board and uses a **Breadth-First Search (BFS)** algorithm for each piece of food to find the shortest valid path to each one. Then, using this information, it uses a **Greedy algorithm** to compare the paths found and select the path to the food that requires the fewest moves. Finally, the snake makes only the next move along the selected path. After that, the positions of both snakes and the game state are updated, and the process repeats in the next cycle.

2.2 Breadth-First Search (BFS)

The goal of this algorithm is to find the shortest path between the head of the AI-controlled snake and the position of a given piece of food, avoiding positions that are off the board or occupied by the bodies of either snake. It works by using a Queue under the First-In First-Out (FIFO) principle to check available positions, also keeping track of visited positions and the origin of each move.

Next, its pseudocode is shown.

```
Algorithm BFS(start, target, obstacles)
```

```
    // Input: Starting position, target food position and blocked positions
    // on the board
```

```
    // Output: Shortest valid path from start to target, or NO_PATH if no
    // path exists
```

```
    Queue ← empty queue
```

```
    Visited ← empty set
```

```
    Parent ← empty map
```

```
    ENQUEUE(Queue, start)
```

```
    ADD(Visited, start)
```

```
    Parent[start] ← NONE
```

```
    while Queue is not empty do
```

```
        current ← DEQUEUE(Queue)
```

```
        if current = target then
```

```
            return RECONSTRUCTPATH(Parent, target)
```

```
        end if
```

```
        for each neighbor of current do
```

```
            if ISVALID(neighbor, obstacles)
```

```

        and neighbor not in Visited then

            ADD(Visited, neighbor)

            Parent[neighbor] ← current

            ENQUEUE(Queue, neighbor)

        end if

    end for

end while

return NO_PATH

```

To better understand the algorithm, each component of it will be explained in detail.

2.2.1. Queue

The queue stores the positions that are found but haven't been processed by the BFS algorithm. It works on the **FIFO** principle, meaning that each position is processed in the order it was added to the queue. The **BFS** algorithm is then applied to these stored positions to find new unprocessed positions. If the queue is empty, the algorithm returns that there is no path to the position where the food is.

2.2.2. Valid neighbor

A valid neighbor is a position next to the current position that is within the board limits, not occupied by either of the two snakes, and hasn't been visited before. Valid neighbors are added to the Queue to continue the search using BFS.

2.2.3. Visited

In order to prevent the AI from processing the same position too many times, each position added to the queue is also recorded in Visited. This way, the algorithm can identify positions that have already been discovered and avoid adding them to the queue again. This reduces unnecessary processing and prevents loops during BFS execution.

2.2.4. Parents

By saving the parents of each position, we have a record of which position the algorithm was in to get to the current one. This record allows us to have a history of positions so that later we can reverse them and build the path the snake should follow to reach the food we want it to get to. This record will be used by the algorithm that will be explained next.

2.3. Road reconstruction

For the AI to be able to decide its next move, it's necessary to reconstruct the path that had to be taken to reach the food position, which was found using the BFS algorithm. To do this, another algorithm is used that works by using the parents of each position to figure out where each move came from and finally reversing it so the AI knows what its next move will be. The pseudocode of the algorithm is as follows.

```

Algorithm RECONSTRUCTPATH(Parent, target)

    // Input: Parent map generated by BFS, target position

```

```

// Output: Path from the starting position to target

Path ← empty list

current ← target

while current ≠ NONE do

    APPEND(Path, current)

    current ← Parent[current]

end while

REVERSE(Path)

return Path

```

2.4. Greedy algorithm and objective selection

To be able to select the closest target from the snake's current position, a greedy algorithm is used that compares the movements needed to reach the different foods found by the BFS algorithm earlier. This ensures that the AI makes the best decision given the current state of the game and the board, and that its next move heads toward the closest possible target. The strategy doesn't aim to plan the entire game, but rather to make a local decision based on the information available in the current cycle. The pseudocode of the algorithm to be applied is as follows.

```

Algorithm GREEDYSELECTFOOD(Head, Foods, Obstacles)

    // Input: Snake head position, available food positions, and blocked
    positions

    // Output: Path to the selected food, or NO_PATH if no food is reachable

    BestPath ← NONE

    for each food in Foods do

        Path ← BFS(Head, food.position, Obstacles)

        if Path ≠ NO_PATH then

            if BestPath = NONE

                or length(Path) < length(BestPath) then

                    BestPath ← Path

            end if

        end if

    end for

    if BestPath ≠ NONE then

        return BestPath

    end if

```

```
return NO_PATH
```

Now then, there are some special cases that are worth mentioning just in case they might happen.

2.4.1. Two or more equal paths

If there are two or more paths with the same number of moves, the Greedy algorithm will choose the first one based on the order in which the foods are stored. Since the AI doesn't foresee future moves and only makes the best decision based on the current state of the board, it considers paths of the same length to have the same cost in the current cycle.

2.4.2. If there are no available paths

If BFS doesn't find any available paths to any of the food, the Greedy algorithm won't have any options to compare. In that case, the AI will make a safe move to avoid an immediate collision. After the game state updates, it will recalculate the possible targets and their paths, without analyzing the player's future moves or planning the game in the long term.

2.4.3. Safe movement

When there is no available path to any food, the AI will evaluate the possible directions and choose a move that doesn't cause an immediate collision.

```
Algorithm SAFEMOVE(Head, CurrentDirection, Obstacles)
// Input: Current head position, current direction and blocked positions

// Output: Safe direction or NO_MOVE

for each direction in [UP, DOWN, LEFT, RIGHT] do

    if direction is not opposite to CurrentDirection then

        neighbor ← Head + direction

        if ISVALID(neighbor, Obstacles) then

            return direction

        end if

    end if

end for

return NO_MOVE
```

2.5. Decision cycle

To wrap up the explanation of the data structures and relevant algorithms, the algorithms and pseudocode are combined to create a logical decision loop, which repeats every time the game state is updated. This update includes new obstacles, the growth of one of the two snakes, or the appearance and disappearance of new food. Initially, BFS is applied along with the algorithm to build the path to each available food. Then, using Greedy, the best decision is made depending on that state, and the game logic takes care of updating the game state and providing the new state.

```

Algorithm DECIDEMOVE(GameState)

// Input: Current game state

// Output: Next movement direction for the AI

Head  $\leftarrow$  GameState.AI.body[0]

Obstacles  $\leftarrow$  GameState.player.body  $\cup$  GameState.AI.body

Path  $\leftarrow$  GREEDYSELECTFOOD(
    Head,
    GameState.foods,
    Obstacles
)

if Path  $\neq$  NO_PATH then
    NextPosition  $\leftarrow$  Path[1]
    return GETDIRECTION(Head, NextPosition)
end if

return SAFEMOVE(
    Head,
    GameState.AI.direction,
    Obstacles
)

```

3. BOARD AND STATE REPRESENTATION

At this point, it's defined how the game state will be represented within the code, what data structures and components will be used, how they will relate to each other, and how this representation will allow integration with the AI's BFS algorithm. The rules for movement, growth, food, and collisions have already been defined earlier, so here it's mainly established how that information will be stored and accessed.

3.1. Board representation

The board will be represented by its dimensions and a coordinate system (x, y). The position of each element will be used to know where it is on the board. The proposed structure is:

Board

```

├── width
└── height

```

width represents the board's width and height represents its height.

The final size of the board is yet **to be decided**. These dimensions will help determine whether a position is within the valid playing area or not.

The information about occupied positions won't be stored as a separate structure; instead, it will be derived from the positions of the snakes defined in the current state.

This setup makes it easy for both the game logic and the AI to check the board's boundaries.

3.2. Snakes representation

To represent the two snakes, a structure called Snake will be used, which will store its body using a list of positions, as well as its direction, score, and current state. The proposed structure will be:

```
Snake
├── body
├── direction
├── score
└── alive
```

The body attribute contains the coordinates of all the segments. The first position in the list corresponds to the head, so it can be obtained by:

```
head = body[0]
```

From this list, it'll be able to identify the head and the tail:

```
body[0] → cabeza
```

```
body[-1] → cola
```

The same snake structure will be used for:

```
player → snake controlled by the player
```

```
AI      → snake controlled by the artificial intelligence
```

This avoids creating two different structures to represent elements that have the same basic characteristics.

3.3. Actual direction and alive/dead state

Each Snake class will store its current direction using the attribute direction.

The current direction and the alive status are part of each snake's information and can be accessed by the game logic, and, when necessary, by the AI module.

The alive attribute will also be stored, which indicates whether the snake is still active.

alive = True, means the snake is still in the game, while:

alive = False, means the snake was eliminated due to a collision.

This way, the Snake structure contains both the information needed to know its position and movement as well as its status in the game.

3.4. Food representation and general state

Each meal will be represented by a Food structure that will store only its current position on the board:

```
Food
└── position
```

During the game, there will be three Food instances active at the same time. These will be stored in a list called foods, which will allow you to check the positions of all the available food.

```
GameState
├── board
├── player
├── AI
├── foods
└── status
```

player and AI correspond to the two instances of Snake, while foods contains the three instances of Food currently available on the board.

Every time a snake eats a food, the corresponding instance is removed from foods and a new food is generated in a valid position. This way, the number of three active foods is kept constant throughout the game.

Additionally, the status attribute will store the general state of the game.

The positions of the foods will be represented by (x, y) coordinates, following the same system used for the snakes. Each food eaten represents one point, and the score will be stored directly within each Snake instance through the score attribute. Every time a snake eats a food, its score will increase by one.

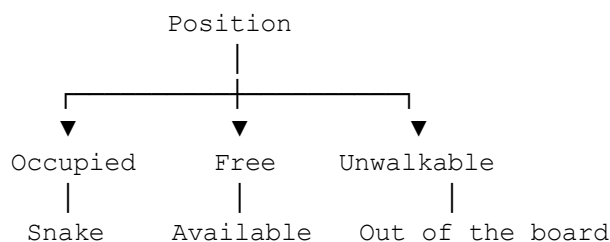
3.5. Identification of occupied, free, and non-passable cells

It won't be necessary to store a separate classification for each cell. This information can be obtained by checking the current game state.

A position will be considered:

- Occupied: if it belongs to the player's or AI's snake body.
- Free: if it is within the board limits and doesn't belong to any snake.
- Unwalkable: if it is outside the board limits.

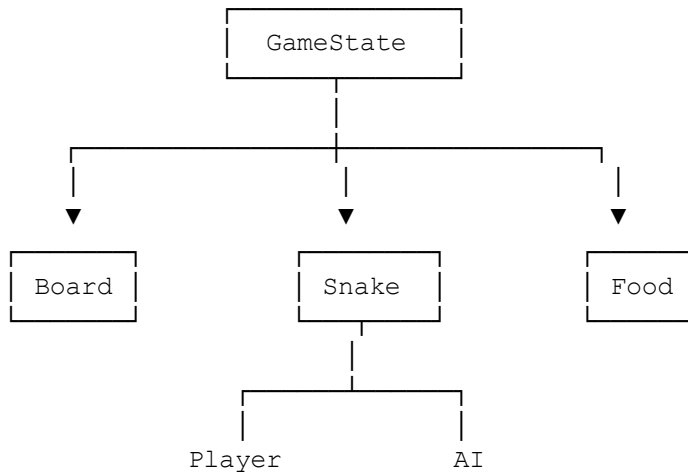
Conceptually:



This way, the availability of a position is determined from GameState and doesn't depend on Pygame.

3.5. Main classes and components

The main components of the architecture will be Board, Snake, Food, GameState, Game, AI, and UI. Their relationship is represented as follows:



Its responsibilities will be:

Components	Responsibility
Board	It keeps the board's dimensions and lets you check if a position is within its limits.
Snake	It represents a snake and its current information.
Food	It keeps the food in the position it's in.
GameState	Centralize the complete state of the game.
Game	Coordinate the logic and state updates during the game.
AI	Calculate the next move of the computer-controlled snake using the GameState information, BFS, and the Greedy algorithm.
UI	Manage player interaction and representation using Pygame to display the game.

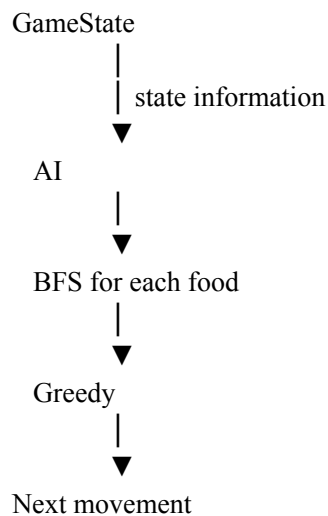
3.6. Communication between the AI and the game state

The AI module will not get information from the image or the elements drawn by Pygame. Instead, it will receive the information stored directly in GameState.

To calculate the movement of the AI-controlled snake, it can check:

- The dimensions of the board.
- The current position of the head of the AI-controlled snake.
- The full body of the AI-controlled snake.
- The full body of the player's snake.
- The positions of the three active foods stored in foods.
- The state of both snakes.
- The current direction of the AI-controlled snake, when needed to determine the next move.
- The overall game state stored in status.

Conceptually:



The AI will use the information from GameState to run BFS toward each of the available foods. Then, the Greedy algorithm will compare the paths found and select the one corresponding to the food that can be reached using the smallest number of moves.

Finally, the AI will return the direction for the next move along the chosen path. If there's no valid path to any food, it will use SAFEMOVE to find a safe move. The Game component will use this direction during the next update cycle.

3.8. Available information for BFS

BFS will use the information available in GameState to build and explore the possible board positions.

The starting point of each search will be the position of the AI-controlled snake's head:

Start = ai.body[0]

Since there are three foods active at the same time, BFS will run once for each of them. In each run, a different food will be used as the target:

For each food in foods:

Objective = food.position

Path = BFS(Start, Objective, Obstacles)

This way, the AI will be able to know the shortest path to each food that is reachable from its current position.

To analyze each position, BFS can check:

- board.width and board.height, to check the boundaries.
- ai.body, to know the positions occupied by its own snake.
- player.body, to know the positions occupied by the player's snake.
- foods, to get the positions of the three available foods.
- food.position, corresponding to the food used as the target in the current BFS run.

Also, in each run BFS internally creates the Visited set, used to record the positions that have already been processed during that search.

From a position (x, y), BFS will be able to evaluate its four neighbors:

(x, y-1) up

(x, y+1) down

(x-1, y) left

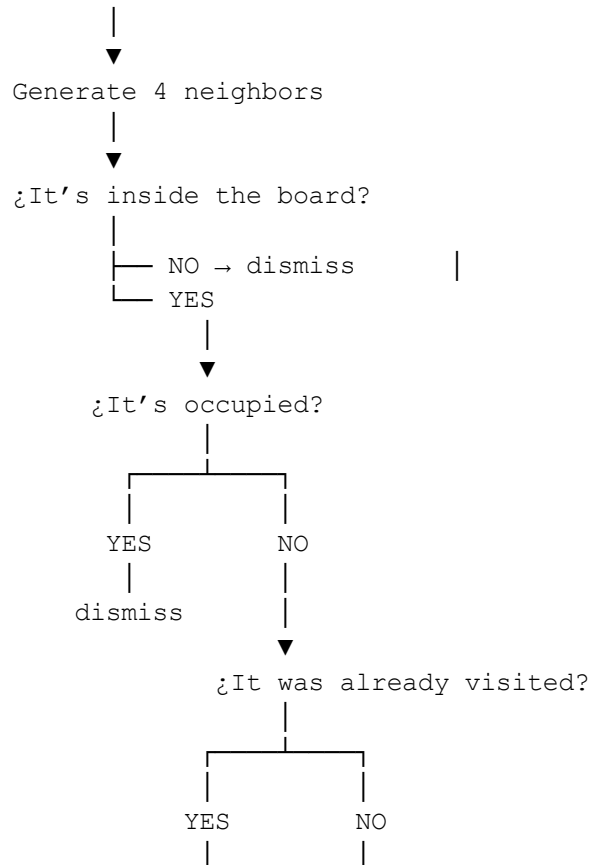
(x+1, y) right

Before adding them to the path, BFS should check:

1. That the position is within the board's dimensions.
2. That it is not a position that is occupied and should be avoided.
3. That it hasn't been visited previously during the current search.

Conceptually:

Actual position



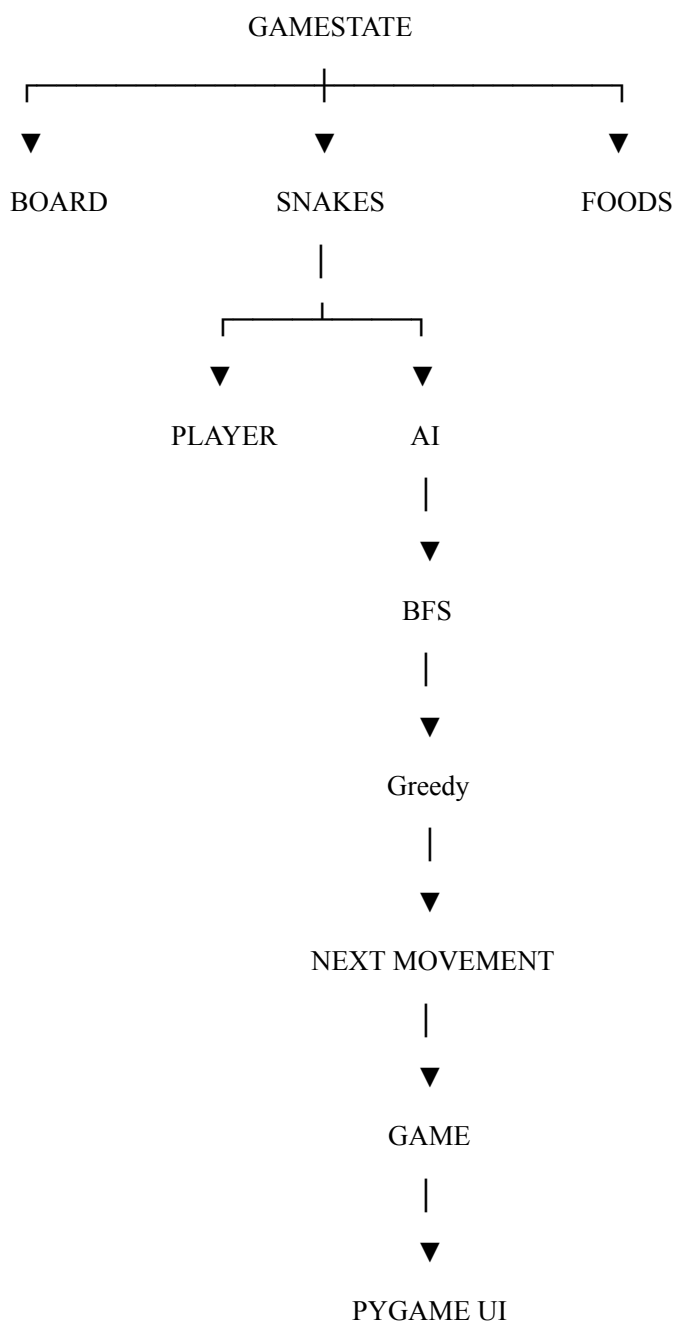
dismiss add

This procedure will be repeated independently for each of the three meals. The valid paths obtained will then be compared by the Greedy algorithm, which will choose the path with the fewest moves.

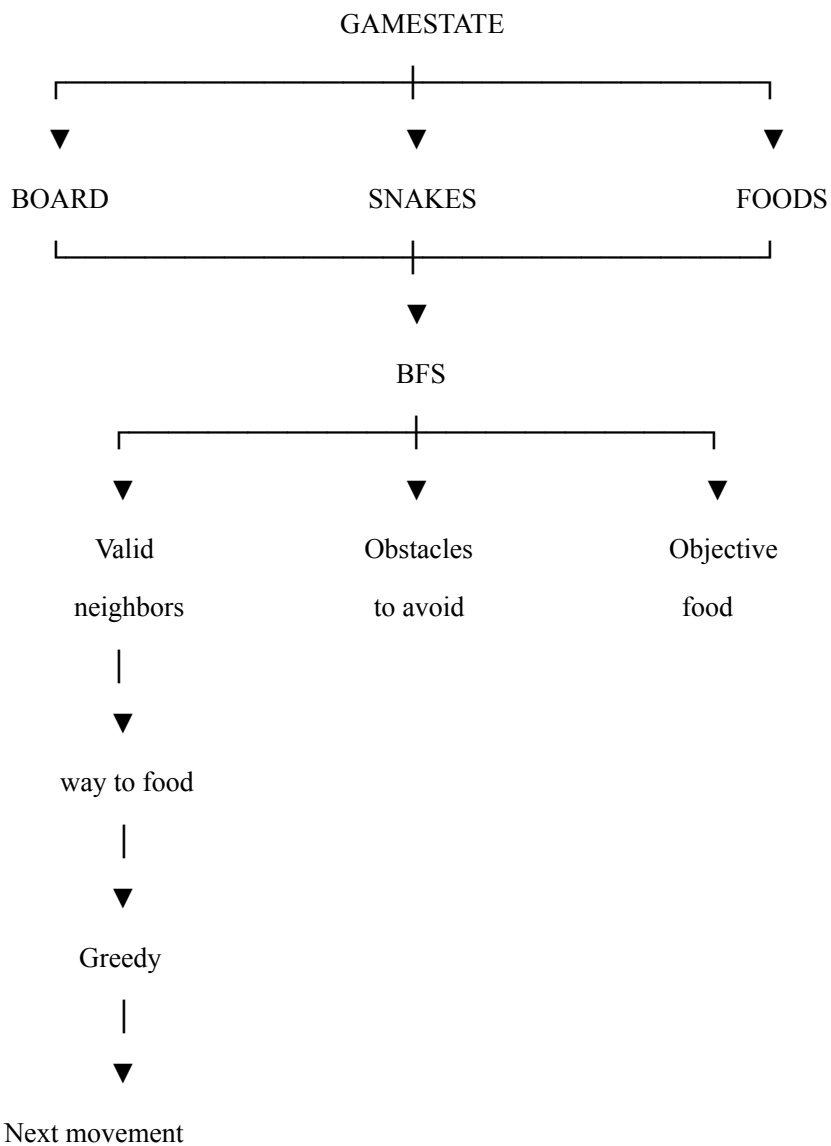
BFS doesn't need to know anything about the Pygame drawing. It only needs to check the logical representation of the state to know where the snakes are, where the food is, what the board boundaries are, and which positions it can move to.

3.9. Summary of architecture and BFS

3.9.1. General architecture



3.9.2. BFS integration



BFS will use the following information and structures during its execution:

Information	For what does BFS use it?
Board.width Board.height	Knowing if a position is inside the board
AI.body	Detect the positions occupied by the AI snake
Player.body	Detect the positions occupied by the player's snake

<code>AI.body[0]</code>	Get the initial BFS position, corresponding to the AI snake's head
<code>foods</code>	Get the three available meals that can be used as targets.
<code>Food.position</code>	Knowing the position of the food (target)
<code>Visited positions</code>	Avoid going over the same positions again

4. PYGAME WIREFRAME

4.1. Explanation of the “Pygame” tool

Pygame will be the tool used to develop the graphical interface of the game. It will allow representing the board, the snakes, the foods, the scores, and the different screens of the game. It will also be used to capture the player's inputs via the keyboard and to manage events such as pause, restart, and closing the application.

4.2. Installation and dependencies of the tool

The game will be developed using Python 3.9 or higher and the Pygame library. No additional external libraries will be used apart from those included in Python's standard library.

4.3. Handling of the different sections of the wire frame

Different frames will be used to separate events such as the start of the game, pause, the winner, and the game itself.

- **Start screen:** It will be the one that appears every time the player starts the game or restarts a match.
- **Pause screen:** It will respond to the P button using events; likewise, there will be a button inside the game in a corner of the screen that will allow this frame to appear.
- **Winner screen:** It will be the one that indicates who won the match, together with a button that will allow restarting the match.
- **Game screen:** The most important of all; this one will not be treated as a frame, but rather will be the one that occupies all the program's space; this one will contain the game and its elements.

Take into account that the game screen will be the only one that will occupy the entirety of the game. The rest of the screens will be used as frames that will be assumed on top of the game screen without occupying the whole screen, occupying the central part. The visible parts of the game screen must darken, for which a base frame will be used that will take a grayish color that allows the background of the same to be seen.

4.4. Development in response to the game logic

This section will have its own divisions for a greater explanation of the implementation.

4.4.1. The frames

The appearance of each frame will respond to a game event and may generate a specific action.

- **Start screen:** will allow starting or restarting a match.
- **Pause screen:** will temporarily stop the updating of the game elements and will deactivate the response to movement controls.
- **Result screen:** will show the final result of the match and will allow restarting it.
- **Game screen:** will respond to the player's actions and to the movements calculated by the AI.

4.4.2. The elements of the match

These elements will be treated as assets that will be stored by default in memory. Among these elements are:

- **The snakes:** it will be composed of two assets. And it will be its head, which must be differentiated by being the reference of the state of the match for the player. Its body will be green squares that represent a whole square in a specific position.
- **The apples:** images without background that will occupy a space on the board; they will have a striking appearance to capture the user's attention.
- **The limits or walls:** They will have a simple blue aesthetic; they will simply serve as indicators that the snake crossed the permitted limits.

4.4.3. Game over

In response to the events defined in the program logic, the result screen will appear superimposed, where automatically the elements of the match will be frozen, and depending on the event the window will indicate a specific winner or, failing that, a tie. At the bottom there will be the restart option on a button.

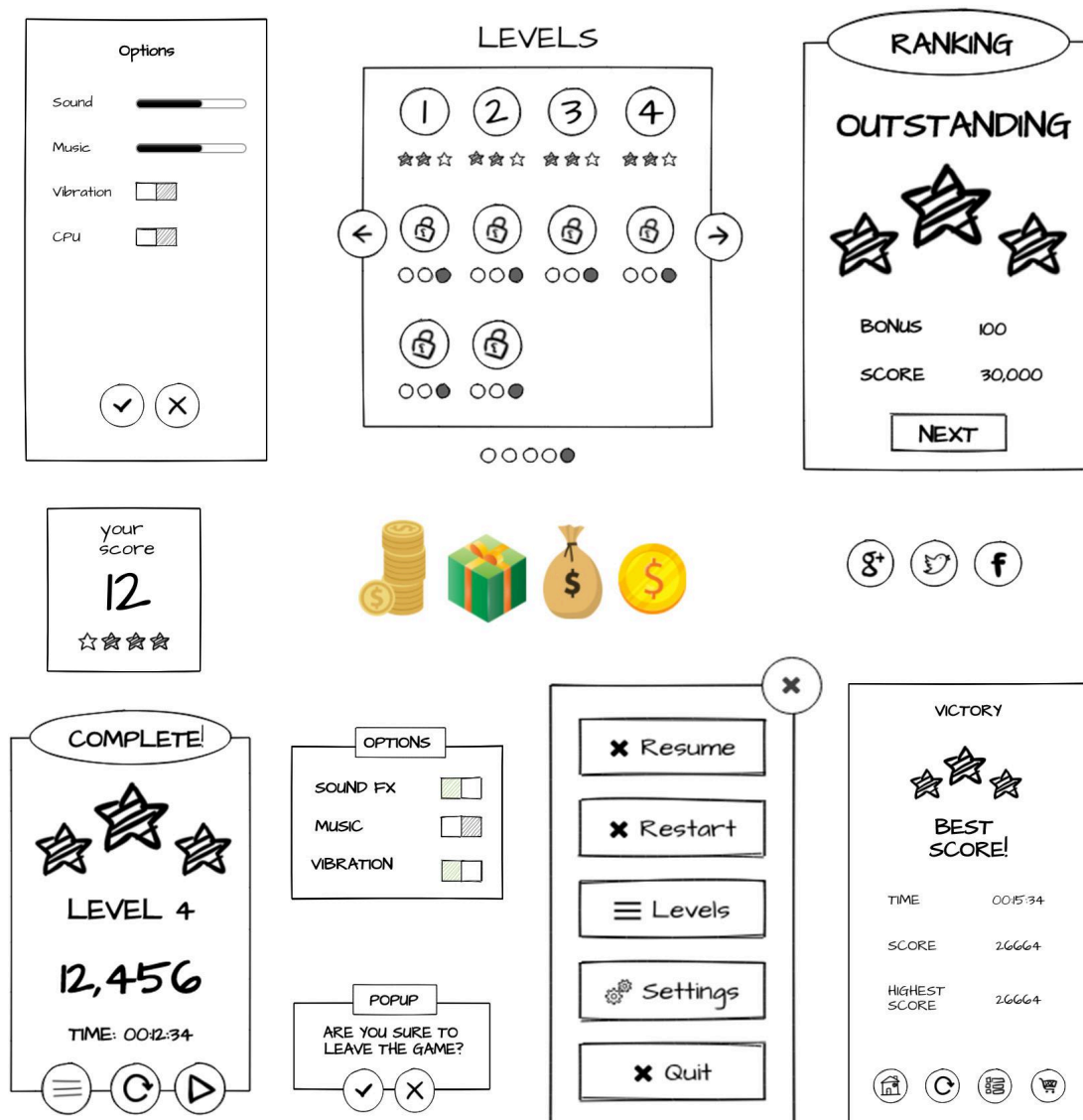
4.5. Additional

From the game interface using Pygame, an attempt will be made to implement music treating sounds as assets of the program, where the following points will be taken into account:

- The music must be focused on accompanying the player's experience.
- There must be different sounds for both the start screen and the game screen.
- When the player pauses the game, the music must stop.
- The player will be able to activate/deactivate from the pause frame, so a frame focused on the game configuration must be created.

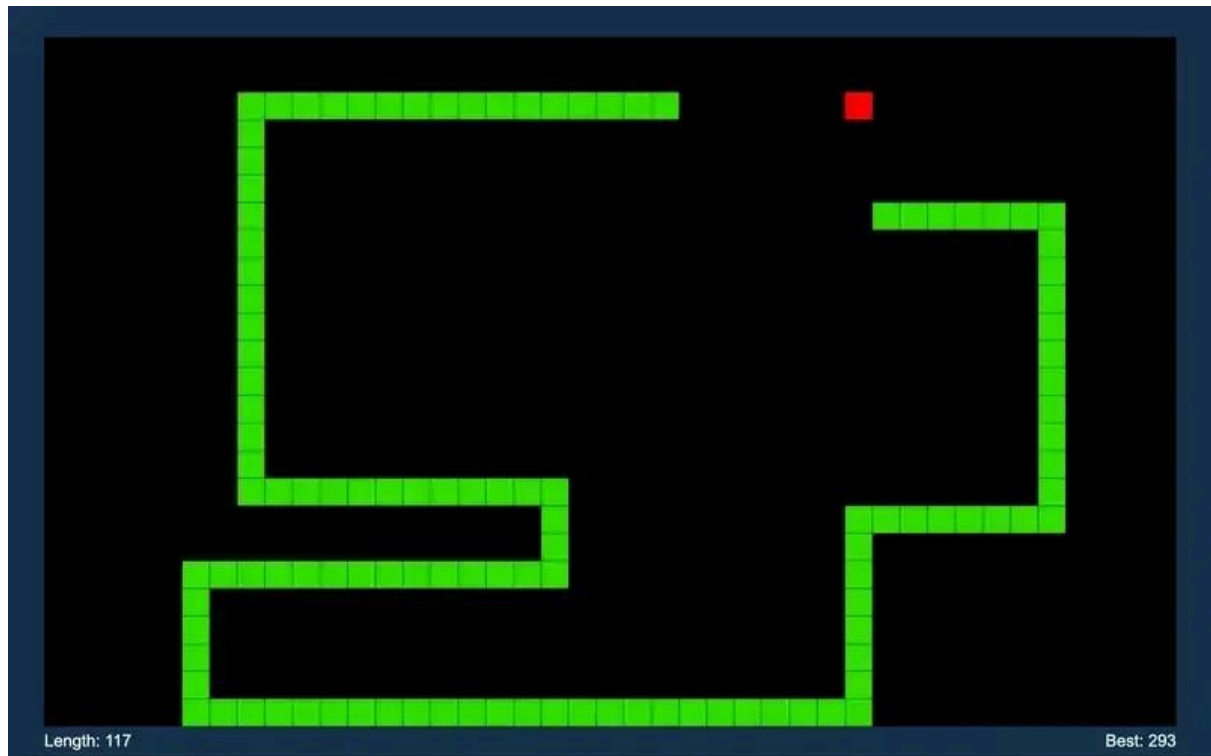
Additionally, the possibility of adding an indicator of the snakes' length at the bottom of the board will be sought, using the walls so as not to obstruct the player.

4.6. Reference images



Reference: Taken from MockFlow.com, the free game-pack for wireframes.

The development of the wireframe will be done taking into account the aesthetics handled in the image extracted from the web, but will not exactly trace its elements, since they will not cover the requirements of the game logic.



Reference: coolmathgames.com, snake game history.

This image will serve as a reference for the creation of the game, showing an interface like the one sought, simple, but at the same time striking tones that capture attention.

5. TASKS DISTRIBUTION STATEMENT

Everyone will have to make a key point in the project, where the duty will be the developing and the part submission.

Member	Main duty	Periodo de trabajo
Miguel Angel Quintero Puentes	Game logic	From Tuesday, September 15 to Thursday, September 17.
Brayan Esteban Bonilla Piñeros	AI, game structure and algorithms development	From Tuesday, September 15 to Sunday, September 20.
Suhayl Mariana Barrantes Alvarado	State representation and architecture	From Tuesday, September 15 to Sunday, September 20.
Luis Felipe Zubieta Rincón	Pygame wireframe and inter	From Tuesday, September 15 to Sunday, September 20.
Todo el equipo	Checking and document integration	Sunday, September 20 and Monday, September 21.

In this way, each member focused on a specific component of the project and afterward a group review was carried out to verify the coherence and integration among the different sections of the document.

6. JUSTIFICATION OF THE DATA STRUCTURES AND THE ALGORITHMS

At this point, the decision-making for the implementation of the data structures and the algorithms that will be used within the game will be justified, emphasizing the decisions that have to do with its AI.

6.1. Breadth-First Search (BFS): Within the game logic, each movement between adjacent positions has the same cost. For this reason, BFS allows finding the path with the fewest number of movements from the current position of the AI to each available food.

6.2. Use of Queue: Queue uses the FIFO principle to process the positions in the order in which they are discovered. This allows BFS to explore the nearest positions first and continue until it finds the target food or exhausts the available positions.

6.3. visited function: In order to prevent the algorithm from processing the same position more than once, the visited label is in charge of telling BFS that that position has already been discovered previously. This helps the algorithm not consume extra resources or time within its execution, in addition to implementing it in a more orderly way within the game's operation.

6.4. Use of parent: The use of Parent allows recording from which position each new position was reached. This information is later used to reconstruct the path from the food to the snake's initial position.

6.5. Greedy Algorithm: The Greedy algorithm is used to make the best local decision according to the current state of the game at that moment. It is in charge of analyzing the options given by the BFS algorithm to select the target that contains the fewest movements to fulfill it. Additionally, the options for making the decision in case certain specific situations occur help the game flow normally.

6.6. Integration of the algorithms: The algorithms are integrated so that the Artificial Intelligence can think and process what its next movement will be starting from the current state in an efficient manner. Additionally, each component present in the algorithms comes together in a logical cycle to optimize the decision-making of the computer-controlled snake.